Comprehensive Study Guide

CME 295: Transformers & LLMs — Lecture 5

Preference Tuning: Data Collection, RLHF, PPO, Best-of-N, DPO

Stanford CME 295 by Afshine & Shervine Amidi · For: Applied AI/ML Engineers

Contents

Topic Map

1. **Why Preference Tuning?**
   - 1.1 SFT produces a helpful model, but not one aligned to human preferences
   - 1.2 Three reasons preference tuning is preferred over expanding SFT
   - 1.3 Negative signal injection: what SFT cannot do
2. **Preference Data Collection**
   - 2.1 Pointwise vs. pairwise vs. listwise preference formats
   - 2.2 Why pairwise is the standard
   - 2.3 Recipe: prompt sourcing, response generation, labeling
   - 2.4 Labeling methods: human rating, LLM-as-judge, rule-based proxies
3. **RLHF: Reinforcement Learning from Human Feedback**
   - 3.1 RL formulation: agent, state, action, policy, reward, environment
   - 3.2 Mapping to LLM setting
   - 3.3 Step 1 — Reward Modeling: Bradley-Terry formulation
   - 3.4 Reward model loss derivation from MLE
   - 3.5 Reward model architecture (decoder-only + classification head)
   - 3.6 Step 2 — RL Training: objective (maximize reward, minimize KL)
   - 3.7 Reward hacking and why KL constraint is necessary
   - 3.8 On-policy vs. off-policy training
4. **PPO: Proximal Policy Optimization**
   - 4.1 Advantage function: reward minus baseline
   - 4.2 Value function: token-level reward estimator
   - 4.3 GAE (Generalized Advantage Estimation)
   - 4.4 PPO-Clip: ratio clipping, ϵ, behavior for $A > 0$ and $A < 0$
   - 4.5 PPO-KL Penalty: old vs. reference model distinction
   - 4.6 Four models required: policy, value, reward model, base
   - 4.7 PPO challenges: hyperparameters, instability, exploration
5. **Best-of-N (BoN)**
   - 5.1 Concept: generate N completions, score all, return the best
   - 5.2 Pros: avoids RL training complexity
   - 5.3 Cons: expensive at inference time; latency scales with N
6. **DPO: Direct Preference Optimization**
   - 6.1 Motivation: supervised alternative to RLHF

6.2 Derivation: PPO objective → optimal policy → reward identification → Bradley-Terry → DPO loss

6.3 DPO loss formula; β interpretation

6.4 "Your LM is secretly a reward model"

6.5 RLHF vs. DPO: ease, models needed, performance

Deep-Dive Notes

Why Preference Tuning?

After SFT, the model is a capable assistant, but may still produce outputs that are factually correct yet undesirable in tone, helpfulness, or safety. Preference tuning is the third training stage that aligns the model's outputs with what humans prefer.

Three Reasons Preference Tuning Supplements SFT

1. **Comparing is easier than generating:** Judging which of two poems is better is far easier than writing a great poem from scratch. This makes preference data cheaper to collect at scale than high-quality SFT demonstrations.
2. **SFT distribution sensitivity:** Adding misbehavior examples to the SFT set risks biasing the model's output distribution. Preference tuning handles negative examples without this risk.
3. **Negative signal injection:** SFT teaches what to generate, not what *not* to generate. Preference tuning explicitly down-weights undesired outputs — something SFT cannot do without rewriting bad outputs into good ones.

Caveat: If the model misbehaves a lot, check the SFT data quality first. Preference tuning cannot fix fundamentally broken SFT.

Preference Data Collection

Data Formats

Three Preference Data Formats

- **Pointwise:** Assign a score (e.g., 0.1–1.0) to each (prompt, response) pair independently. Hard for humans — no reference frame, scale is ambiguous.
- **Pairwise:** Given two responses to the same prompt, indicate which is better (binary: $y_w > y_l$ or $y_w < y_l$). Easy for humans; the standard in practice.
- **Listwise:** Rank N responses from best to worst. More information per annotation but harder and more expensive to produce.

Industry standard: pairwise binary preference.

Data Collection Recipe

1. **Prompt sourcing:** Sample prompts from real user logs or from a curated reference distribution representing the target use cases.
2. **Response generation:** Generate two responses per prompt using the SFT model with temperature > 0 (to get diversity). Alternatively: use synthetic generation or manually rewrite bad responses into good ones.
3. **Labeling:** Annotators (human or LLM) compare the two responses and mark which is preferred.
   - **Human rating:** Gold standard for RLHF (the "HF" in RLHF). Expensive but most reliable if guidelines are clear.
   - **LLM-as-judge:** Use a strong LLM to compare responses. Cheaper, but introduces model bias.
   - **Rule-based proxies:** BLEU, ROUGE, factuality checkers. Less common today.

Annotation Guideline Quality

Human preference labels are only as good as the guidelines given to annotators. Vague guidelines produce noisy, inconsistent labels. Preference tuning propagates this noise into the model. Reward model quality is directly bounded by annotation quality.

RLHF: Reinforcement Learning from Human Feedback

RL Formulation

RL Basics

- **Agent:** the entity taking actions.
- **State** s_t: the current situation of the agent.
- **Action** a_t: what the agent does at time t.
- **Policy** $\pi_\theta(a_t|s_t)$: probability of action a_t given state s_t. Parameterized by θ.
- **Reward** r_t: feedback signal after the action.
- **Environment:** the world the agent acts in.

Goal: learn θ that maximizes expected cumulative reward.

RL Formulation for LLMs

RL Concept	LLM Equivalent
Agent	The LLM
State s_t	Input tokens processed so far
Action a_t	Next token to generate
Environment	Vocabulary (set of all tokens)
Policy $\pi_\theta(a_t\|s_t)$	Output probability distribution of the LLM
Reward	Score from reward model (per full completion)

RLHF vs. RLAIF

RLHF (Reinforcement Learning from Human Feedback): preference labels come from humans. **RLAIF** (RL from AI Feedback): preference labels come from another LLM. RLAIF is cheaper and scales better but can inherit the labeling model's biases.

Step 1: Reward Modeling

Bradley-Terry Formulation (Bradley & Terry, 1952)

The probability that response y_i is preferred over response y_j, given prompt x:

$$P(y_i \succ y_j \mid x) = \frac{e^{r(x,y_i)}}{e^{r(x,y_i)} + e^{r(x,y_j)}} = \sigma\big(r(x,y_i) - r(x,y_j)\big)$$

where $r(x,y)$ is the **reward score** for (prompt, response) pair, and σ is the sigmoid function.

Reward Model Loss: Derivation from MLE

Given a dataset of n pairwise preference pairs $\{(x^{(k)}, y_w^{(k)}, y_l^{(k)})\}_{k=1}^n$ where y_w is the winning response and y_l is the losing response:

Step 1 — Likelihood: Assuming i.i.d. pairs, maximize:

$$\prod_{k=1}^{n} P(y_w^{(k)} \succ y_l^{(k)} \mid x^{(k)}) = \prod_{k=1}^{n} \sigma\big(r(x, y_w) - r(x, y_l)\big)$$

Step 2 — Log-likelihood (log sum is more stable than product):

$$\sum_{k=1}^{n} \log \sigma\big(r(x, y_w) - r(x, y_l)\big)$$

Step 3 — Maximize by minimizing negative log-likelihood:

$$\boxed{\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma\big(r(x, y_w) - r(x, y_l)\big)\Big]}$$

Intuition: This loss pushes $r(x, y_w) \to +\infty$ and $r(x, y_l) \to -\infty$, ensuring the sigmoid (probability of preferring y_w) approaches 1.

Reward Model Architecture

The reward model takes the full (prompt, response) as input and outputs a scalar score. Two options:

- **Decoder-only LLM + linear head:** Project the final token's hidden state to a single scalar. Most common today.
- **Encoder-only (BERT-like) + CLS projection:** Bidirectional encoding, less common.

Key: The reward model is trained *pairwise* but operates *pointwise* at inference time — it takes one (prompt, response) and returns one score.

Step 2: RL Training (The RLHF Loop)

RLHF RL Training Objective

Train the policy π_θ (the LLM) to maximize expected reward while not deviating too far from the reference model π_{ref} (the SFT model):

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\Big[r(x, y) - \beta \cdot D_{\text{KL}}\big(\pi_\theta(\cdot|x) \,\|\, \pi_{\text{ref}}(\cdot|x)\big)\Big]$$

- $r(x, y)$: reward model score for the generated response.
- D_{KL}: KL divergence, measuring how much π_θ has drifted from π_{ref}.
- β: trade-off hyperparameter ($\beta \approx 0.1$–0.5).
- **Reward model is frozen**; only the LLM weights θ are updated.

KL Divergence

$$D_{\text{KL}}(P\|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Measures how different distribution P is from Q. Always ≥ 0 (proof via Jensen's inequality on the concave log function). Equals 0 if and only if $P = Q$.

Reward Hacking

If the policy is optimized too aggressively for the reward signal, it may find responses that score highly on the reward model but violate the true underlying goal. Example: if reward is correlated with response length, the policy learns to generate verbose padding. The KL constraint and the fact that the reward model is imperfect (trained on limited human data) make reward hacking a real concern. The KL penalty prevents this by keeping the policy close to the well-behaved SFT model.
Three reasons for the KL constraint:
1. Preserve the knowledge and capabilities of the base SFT model.
2. Prevent exploitation of reward model imperfections.
3. Maintain training stability.

On-Policy vs. Off-Policy Training

On-policy: At each training step, the model generates responses using its *current* policy, then receives rewards for those responses and updates. RL-based preference tuning (PPO, GRPO) is on-policy.
Off-policy: Training uses data generated by a *different* policy (e.g., a previous model version or a fixed dataset). SFT is off-policy (uses fixed demonstration data). DPO is also effectively off-policy.
On-policy training is more expensive (requires live generation) but avoids distribution shift between the training data and the current model.

PPO: Proximal Policy Optimization

Advantage Function

Advantage Function

Instead of optimizing raw reward r, PPO optimizes the **advantage** A:

$$A \approx r - V(s)$$

where $V(s)$ is the **value function** — an estimate of the expected reward from state s under the current policy. The advantage measures *how much better the actual outcome was than expected.*
Using advantages instead of raw rewards reduces variance and stabilizes training.

Value Function

A token-level estimator that takes the prompt plus partial generation as input and predicts the expected final reward if generation continues under the current policy:

$$V(x, y_{<t}) \approx \mathbb{E}_{\pi_\theta}\Big[r(x, y) \,\Big|\, x, y_{1:t-1}\Big]$$

Key properties:
- Token-level (not sequence-level like the reward model).
- Trained jointly with the policy during the RL loop (as a regression head).
- Label at each token = eventual full-sequence reward.

The specific method for computing advantages from the value function is **GAE (Generalized Advantage Estimation, Schulman et al., 2015)**.

PPO-Clip

PPO-Clip Objective

Define the **probability ratio**:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

where $\pi_{\theta_{\text{old}}}$ is the policy from the *previous RL iteration* (not the SFT base model). The PPO-Clip objective to *maximize* is:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t\big)\Big]$$

where $\varepsilon \approx 0.1$–0.2 is the clipping hyperparameter.

PPO-Clip Intuition: Case Analysis

When $A_t > 0$ (the action was better than expected — reinforce it):

- We want to increase r_t (make a_t more likely).
- But clip at $1+\varepsilon$ to prevent excessively large updates.
- Shape: linearly increasing for $r_t \in [0, 1+\varepsilon]$, then flat.

When $A_t < 0$ (the action was worse than expected — discourage it):

- We want to decrease r_t (make a_t less likely).
- But clip at $1-\varepsilon$ to prevent excessively large updates in the other direction.
- Shape: linearly decreasing for $r_t \in [1-\varepsilon, \infty]$, then flat.

The clipping prevents any single update from moving the policy too far from where it was.

PPO-KL Penalty

PPO-KL Penalty Objective

$$L^{\text{KL}}(\theta) = \mathbb{E}_t\Big[r_t(\theta)A_t - \beta \cdot D_{\text{KL}}\big(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta\big)\Big]$$

In modern LLM alignment, "old" is replaced by the reference model π_{ref} (the SFT checkpoint):

$$L^{\text{KL}}(\theta) = \mathbb{E}_t\Big[r_t(\theta)A_t - \beta \cdot D_{\text{KL}}\big(\pi_{\text{ref}} \,\|\, \pi_\theta\big)\Big]$$

In practice, many implementations mix both clip and KL terms.

Terminology: Old vs. Reference

- $\pi_{\theta_{\text{old}}}$ = policy from the previous RL iteration. Used in PPO-Clip to prevent large step-to-step updates. Stability concern.
- π_{ref} = the SFT model, frozen throughout RL. Used in the KL penalty to prevent catastrophic forgetting and reward hacking.

These are two different models serving two different purposes.

Four Models in PPO

Models Required for PPO

1. **Policy (π_θ):** The LLM being trained. Generates completions.
2. **Value model (V_ϕ):** Trained jointly with the policy to estimate expected reward per token.
3. **Reward model (RM):** Frozen. Trained in Step 1. Scores (prompt, completion) pairs.
4. **Reference model (π_{ref}):** The SFT model, frozen. Used to compute KL penalty.

For a 70B-parameter model: running four 70B-parameter models simultaneously requires enormous memory — a major practical limitation.

PPO Challenges

PPO Practical Challenges

1. **Two-stage process:** Must first train reward model, then do RL. If either has problems, re-run both.
2. **Many hyperparameters:** β (KL weight), ε (clip), GAE parameters λ and γ, etc.
3. **Training instability:** On-policy updates can be unstable. Clipping and KL penalties help but don't eliminate instability.
4. **Monitoring difficulty:** Unlike SFT (where cross-entropy loss is meaningful), monitoring average reward is noisy and not a clean convergence signal.
5. **Exploration requirement:** The policy must generate diverse completions during training (using sufficient temperature) to learn from both good and bad outputs.
6. **Memory cost:** Four large models required simultaneously.

Best-of-N (BoN)

Best-of-N (BoN)

An inference-time alternative to RL training. Given a prompt:

1. Generate N responses from the SFT model (using temperature > 0).
2. Score each response using the reward model.
3. Return the highest-scoring response to the user.

No additional training of the LLM is required.

BoN Example with $N = 3$

Prompt: "Suggest a new activity I could do with my teddy bear."
Response A: "Watch a movie together." $\rightarrow$ Reward $= +0.8$ (best) (selected)
Response B: "Don't spend time with your teddy bear." $\rightarrow$ Reward $= -2.0$
Response C: "Take your teddy bear on a picnic." $\rightarrow$ Reward $= +0.2$

BoN Limitations

- **Inference cost:** N forward passes per query instead of 1. $N = 4$ means $4\times$ compute cost.
- **Latency:** Even with parallelism, latency $\approx$ max(latency of N completions), which is higher than a single completion (since the max of N samples from a distribution shifts right).
- **Model quality floor:** If the SFT model consistently generates bad outputs, no amount

of N helps.
- **Scale irrelevance:** The absolute scale of reward model scores doesn't matter for BoN — only the ranking matters. (Contrast with RL, where absolute values affect loss magnitude.)

DPO: Direct Preference Optimization

Motivation

DPO Motivation (Rafailov et al., 2023)

DPO eliminates the reward modeling stage entirely. Instead of: (1) train a reward model, then (2) use RL to optimize the LLM — DPO directly optimizes the LLM weights using a single supervised loss applied to preference pairs.
Title of the paper: "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." This refers to the fact that the implicit reward in DPO is expressed entirely in terms of the policy itself.

DPO Derivation

DPO Loss: 5-Step Derivation

Step 1 — Start from PPO objective:

$$\max_{\pi_\theta} \mathbb{E}_{y\sim\pi_\theta}\Big[r(x,y)\Big] - \beta D_{\text{KL}}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)$$

Step 2 — Derive the optimal policy: Solving this optimization analytically (no approximation), the optimal policy is:

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y|x)\exp\left(\frac{r(x,y)}{\beta}\right)$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y|x)\exp(r(x,y)/\beta)$ is the partition function.
Step 3 — Rearrange to express reward in terms of policy:

$$r(x,y) = \beta\log\frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta\log Z(x)$$

Step 4 — Plug into Bradley-Terry formulation:

$$P(y_w \succ y_l \mid x) = \sigma(r(x,y_w) - r(x,y_l))$$

The $Z(x)$ terms cancel (same for y_w and y_l):

$$P(y_w \succ y_l \mid x) = \sigma\left(\beta\log\frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

Step 5 — Maximize log-likelihood over preference data:

$$\boxed{\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]}$$

DPO Loss Interpretation

The DPO loss:

- **Increases** the likelihood of winning responses y_w relative to the reference model.
- **Decreases** the likelihood of losing responses y_l relative to the reference model.
- β controls how strongly to deviate from the reference model. Higher β = stays closer to π_{ref}; lower β = larger updates toward preferences.
- **No reward model required.** The implicit reward is $\hat{r}(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$.
- **Only two models needed:** π_θ (trained) and π_{ref} (frozen SFT model). Compare to PPO's four.

RLHF vs. DPO Comparison

Table 1: RLHF (PPO) vs. DPO Comparison

Dimension	RLHF / PPO	DPO
Training stages	2 (RM then RL)	1 (direct supervision)
Models in memory	4	2
Reward model needed	Yes	No
Training type	On-policy	Off-policy (supervised)
Hyperparameters	Many (β, ε, GAE params, etc.)	Fewer (β mainly)
Stability	Harder to tune	More stable
Distribution shift	Less (on-policy)	More (off-policy)
Peak performance	Generally higher	Slightly lower

Practical Guidance

- Use **DPO** when you want a quick, stable, well-understood implementation. Good for most applied use cases.
- Use **PPO/RL** when you need maximum performance, have RL expertise, and can afford the compute and engineering overhead.
- Performance difference is task-dependent. Xu et al., 2024 provides a comprehensive comparison.

Related Concepts You Should Also Know

1. **GRPO (Group Relative Policy Optimization, DeepSeek, 2024):** An RL variant that eliminates the value function by computing advantages from the *relative* rewards within a group of completions for the same prompt. Averages the group's rewards as the baseline. Reduces from 4 models (PPO) to 3. Particularly effective for reasoning tasks. Will be covered in Lecture 6.
2. **REINFORCE:** A simpler policy gradient algorithm that uses the raw reward as the advantage (no value function). High variance but minimal memory requirements. Monte Carlo return as baseline. Predecessor to PPO.
3. **InstructGPT (Ouyang et al., 2022):** The paper that introduced the RLHF framework for LLMs. GPT-3 was fine-tuned with SFT on human demonstrations, then RLHF with PPO on human preference labels. This became the template for ChatGPT, Claude, and others.
4. **Constitutional AI (Anthropic, 2022):** A variant of RLAIF where the AI critiques and revises its own outputs against a set of principles (a "constitution"), generating synthetic preference data without human labeling.
5. **Implicit Reward in DPO:** The DPO implicit reward $\hat{r}(x, y) = \beta \log[\pi_\theta(y|x)/\pi_{\text{ref}}(y|x)]$ measures how much the trained policy favors response y relative to the reference model. This is the "secretly a reward model" insight.
6. **RewardBench:** A benchmark for evaluating reward models across dimensions: chat, safety, reasoning, instruction-following. Analogous to MMLU for reward models.
7. **KL Divergence Non-Symmetry:** $D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P)$. In PPO/DPO, $D_{\text{KL}}(\pi_{\text{ref}}\|\pi_\theta)$ and $D_{\text{KL}}(\pi_\theta\|\pi_{\text{ref}})$ have different behaviors. The direction matters for how the penalty penalizes out-of-distribution behavior.
8. **Reward Model Dimensions:** Reward models can be trained for specific dimensions: helpfulness, harmlessness, honesty, factuality, safety, tone/friendliness. Multiple specialized reward models can be combined during RL training.
9. **Iterative DPO / Online DPO:** A variant where the policy generates new completions at each training iteration, which are then scored by a reward model and used to form new preference pairs. Combines the distribution-match advantage of on-policy training with DPO's simplicity.
10. **IPO (Identity Preference Optimization):** A variant of DPO that regularizes more strongly to prevent overfitting to the preference data. Addresses cases where DPO can over-optimize.

Build Projects

1. **DPO Fine-Tuning Pipeline** *(5–8 hours, very high signal)*
   Using TRL (HuggingFace's Transformer Reinforcement Learning library), implement DPO fine-tuning on a public preference dataset (e.g., Anthropic's HH-RLHF, or UltraFeedback). Start from a QLoRA-fine-tuned base (you already built this in Lecture 4's project). Train with $\beta \in \{0.05, 0.1, 0.5\}$ and observe the effect on: (a) the implicit reward margin $\hat{r}(y_w) - \hat{r}(y_l)$ on a held-out validation set, (b) generation quality (human eval or GPT-4 judge), (c) KL divergence from the reference model. This directly extends your Finetune-Pipeline work with a state-of-the-art alignment technique.

2. **Reward Model from Scratch** *(4–6 hours, high signal)*
   Train a reward model using the Bradley-Terry loss on a small preference dataset. Start from a 7B base model, add a linear head on the final token's hidden state, and train with the pairwise preference loss. Evaluate on RewardBench. Visualize the score distribution for winning vs. losing responses to verify the training has converged correctly.

3. **Best-of-N vs. DPO Comparison** *(3–4 hours, good signal)*
   For a fixed set of prompts: (a) run BoN with $N \in \{1, 4, 16, 64\}$ using your reward model to pick the best response; (b) run DPO-fine-tuned model with a single generation. Compare quality (reward model score on held-out set) vs. inference compute (number of forward passes). Plot the Pareto frontier of quality vs. compute. This demonstrates the engineering trade-off between training cost and inference cost.

Explain It Back

1. You are explaining RLHF to a product manager. Explain: (a) what a reward model is and what it's trained on, (b) why the RL training loop is necessary after you already have the reward model, (c) why the KL penalty prevents the model from becoming extremely sycophantic or unsafe, using the "lecture applause" metaphor from class.
2. Derive the DPO loss from the PPO objective. State clearly at each of the 5 steps what you are doing and why. Particular focus: why do the $Z(x)$ partition function terms cancel in Step 4?
3. A teammate proposes using BoN with $N = 100$ as a cheap alternative to DPO training, arguing "we don't need to train anything." Walk through both the latency argument (even with infinite parallel compute, why is latency higher?) and the cost argument at scale (1M queries/day $\times$ 100 generations each).
4. Explain the difference between the advantage function and the reward in PPO. Why is maximizing advantage more stable than maximizing raw reward? What role does the value function play?

Interview Practice Questions & Answers

L1 — What is reward hacking and why does the KL penalty prevent it?

Answer: Reward hacking occurs when the policy learns to exploit weaknesses in the reward model — generating outputs that score highly on the metric without actually being good. The reward model is trained on limited human preference data and is imperfect; any scoring function can be gamed by an optimizer. The KL divergence penalty $\beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ penalizes the policy for deviating too far from the SFT reference model. Since the SFT model's behavior is generally well-aligned (having been trained on high-quality demonstrations), staying close to it prevents the policy from finding adversarial outputs that exploit reward model imperfections. Higher β = stronger constraint = less reward hacking risk but also less alignment improvement.

L1 — What is the difference between RLHF and RLAIF?

Answer: In RLHF (Reinforcement Learning from Human Feedback), the preference labels used to train the reward model come from human annotators comparing pairs of responses. In RLAIF (RL from AI Feedback), the labels come from another AI model (typically a stronger LLM acting as judge). RLAIF is cheaper and scales better since humans are expensive and slow. However, RLAIF can inherit biases from the judging model and may not reflect true human preferences. Constitutional AI (Anthropic) is a notable RLAIF approach.

L2 — Derive the reward model loss function from first principles.

Answer: We have pairwise preference data: for each prompt x and pair (y_w, y_l), y_w is preferred. Using Bradley-Terry: $P(y_w \succ y_l) = \sigma(r(x, y_w) - r(x, y_l))$. To find the best reward function parameters θ_r, we maximize the likelihood of observing the preference data:

$$\max_{\theta_r} \prod_k \sigma(r_{\theta_r}(x^{(k)}, y_w^{(k)}) - r_{\theta_r}(x^{(k)}, y_l^{(k)}))$$

Taking the log (to avoid numerical underflow and to convert product to sum), then negating (to convert maximization to minimization):

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma(r(x, y_w) - r(x, y_l))\Big]$$

This is a binary cross-entropy loss where the positive class is the winning response and the model must assign it a higher reward than the losing response.

L2 — Explain the PPO-Clip objective. What is $r_t(\theta)$ and why clip it?

Answer: $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the ratio of the probability of action a_t under the new policy vs. the previous RL iteration's policy (not the SFT model). When $r_t > 1$, the new policy is more likely to take that action; when $r_t < 1$, less likely. The PPO objective clips r_t to the range $[1-\varepsilon, 1+\varepsilon]$, preventing any single update from moving the policy too far. This serves two purposes: (1) training stability, since large probability shifts can cause instability; (2) preventing policy collapse, where the model gets trapped in a degenerate local optimum by taking one too-large step.

L3 — Derive the DPO loss starting from the PPO objective. What is the key insight?

Answer: Start from the PPO objective: $\max_\pi \mathbb{E}[r(x,y)] - \beta D_{\text{KL}}(\pi \| \pi_{\text{ref}})$. Solving analytically gives the optimal policy $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x)\exp(r(x,y)/\beta)$. Rearranging: $r(x,y) = \beta \log[\pi^*(y|x)/\pi_{\text{ref}}(y|x)] + \beta \log Z(x)$. Plugging into the Bradley-Terry probability $P(y_w \succ y_l) = \sigma(r(x,y_w) - r(x,y_l))$, the $Z(x)$ terms cancel (they appear in both $r(x,y_w)$ and $r(x,y_l)$). The key insight: the reward can be expressed purely as a function of the policy and the reference model, with no separate reward model needed. Maximizing the Bradley-Terry likelihood over preference pairs gives the DPO loss: $\mathcal{L}_{\text{DPO}} = -\mathbb{E}[\log \sigma(\beta \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)})]$.

L2 — What is the value function in PPO and why does it need to be trained?

Answer: The value function $V(x, y_{<t})$ estimates the expected future reward from a partial sequence under the current policy. It's used to compute the advantage $A \approx r - V$. Training is necessary because the value function must track the current policy — as the policy changes during RL training, what counts as a "good" expected reward from any given partial state also changes. A stale value function would give inaccurate baselines, increasing variance and slowing training. The value function is typically implemented as a regression head on top of the LLM, sharing the backbone, and is trained with the policy simultaneously using the reward model's scores as labels.

L1 — Why does pairwise preference annotation produce a pointwise reward model?

Answer: During training, the reward model sees pairs (y_w, y_l) and optimizes the pairwise Bradley-Terry loss that involves both scores $r(x,y_w)$ and $r(x,y_l)$. However, the loss gradient updates a single shared model that maps any (prompt, response) pair to a single scalar. The model learns to assign high scores to winning responses and low scores to losing responses — a pointwise operation. At inference time, you simply pass one (prompt, response) pair and get one score. The pairwise structure is in the training signal, not in the model's architecture or inference behavior.

Self-Assessment Test

Scoring: 16–20 = solid mastery; 12–15 = revisit weak spots; <12 = re-study.

Multiple Choice (1 point each)

Q1. The Bradley-Terry formulation models: (a) The probability that response y_i is correct (b) The probability that y_i is preferred over y_j as $\sigma(r_i - r_j)$ (c) The KL divergence between two policy distributions (d) The advantage of taking action a_t

Q2. In the RL formulation for LLMs, what is the "action"?
(a) The full generated response (b) The next token to generate (c) The reward model score (d) The prompt

Q3. In DPO, how many models need to be in memory during training?
(a) 1 (b) 2 (c) 3 (d) 4

Q4. The KL divergence $D_{\text{KL}}(P\|Q)$ is: (a) Always negative (b) Always positive or zero (c) Symmetric ($= D_{\text{KL}}(Q\|P)$) (d) Maximized when $P = Q$

Q5. In PPO-Clip, $r_t(\theta)$ represents:
(a) The reward from the reward model (b) The ratio $\pi_\theta(a_t|s_t)/\pi_{\text{ref}}(a_t|s_t)$ (new over SFT model) (c) The ratio $\pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ (new over previous RL iteration) (d) The advantage function

Q6. Which of these is an advantage of Best-of-N over RL training?
(a) Lower inference compute cost (b) No RL training overhead; simpler implementation (c) Better model quality (d) No need for a reward model

Q7. DPO is considered "off-policy" because:
(a) It uses a frozen reward model (b) The preference data was generated by the SFT model, not the current policy being trained (c) It doesn't use any policy at all (d) The KL divergence is computed against the old model

Q8. The value function in PPO is trained to predict:
(a) The probability of the next token (b) The expected final reward from a partial sequence under the current policy (c) The preference between two responses (d) The KL divergence from the reference model

Short Answer (1.5 points each)

Q9. Write the reward model loss function (derived from MLE on Bradley-Terry) and label every component.

Q10. State the DPO loss and explain what β controls.

Q11. Explain reward hacking with a concrete example. How does the KL penalty prevent it?

Q12. What are the three reasons preference tuning is used instead of simply adding misbehavior examples to the SFT data?

Q13. In PPO, what is the difference between $\pi_{\theta_{\text{old}}}$ (old policy) and π_{ref} (reference model)? What purpose does each serve?

Q14. Why does the Best-of-N approach increase latency even with unlimited parallel compute?

Scenario (2 points)

Q15. You are building the alignment pipeline for a customer service LLM. You have already completed SFT. Your team is deciding between PPO-based RLHF and DPO. Your constraints: 2 A100-80GB GPUs, no RL expertise on the team, 10K human-labeled preference pairs, and a 4-week timeline. You also need to quantify alignment quality. Describe: (a) which approach you choose and why, (b) the full DPO or RLHF pipeline you would run, (c) how you would collect and evaluate the preference data, (d) what metric you use to monitor training.

Answer Key

Q1. (b) Bradley-Terry models $P(y_i \succ y_j) = \sigma(r_i - r_j)$.

Q2. (b) The next token to generate. The full response emerges from many such actions.

Q3. (b) 2 models: the trained policy π_θ and the frozen reference π_{ref}.

Q4. (b) $D_{\text{KL}}(P\|Q) \geq 0$ always (Jensen's inequality); equals 0 iff $P = Q$. It is not symmetric.

Q5. (c) $r_t(\theta) = \pi_\theta / \pi_{\theta_{\text{old}}}$, the ratio between the current and the *previous RL iteration's* policy, not the SFT model.

Q6. (b) BoN skips RL training entirely, requiring no RL engineering. However, it's more expensive at inference time.

Q7. (b) DPO trains on preference data generated by the SFT model, which is a different (older) policy from the one being updated. This is the definition of off-policy.

Q8. (b) The value function estimates expected final reward from any partial sequence state, used to compute the advantage.

Q9. $\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\big(\underbrace{r(x,y_w)}_{\text{winning score}} - \underbrace{r(x,y_l)}_{\text{losing score}}\big)\right]$. The σ is the sigmoid; $r(x,y)$ is the scalar output of the reward model for (prompt x, response y); y_w and y_l are the winning and losing responses in the pairwise preference pair.

Q10. $\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$. β is the regularization strength: higher β forces the trained policy to stay closer to the reference model (more conservative alignment); lower β allows larger updates toward the preferences.

Q11. Example: a reward model trained to reward helpfulness correlates helpfulness with response length. The policy then learns to pad responses with verbose filler to increase reward, without becoming more helpful. The KL penalty $\beta D_{\text{KL}}(\pi_\theta\|\pi_{\text{ref}})$ prevents this by penalizing large deviations from the SFT model, which doesn't engage in padding.

Q12. (1) Comparison is easier than generation: rating which of two responses is better is cheaper than writing the ideal response from scratch. (2) SFT distribution sensitivity: adding more examples risks biasing the training distribution in unintended ways. (3) Negative signal: SFT can only teach the model what to generate, not what to avoid; preference tuning explicitly down-weights bad outputs.

Q13. $\pi_{\theta_{\text{old}}}$ = the policy from the previous RL update iteration, used in PPO-Clip to prevent excessively large step-to-step updates (stability). π_{ref} = the SFT checkpoint, frozen throughout all of RL training, used in the KL penalty to prevent catastrophic forgetting and reward hacking.

Q14. Even with all N responses generated in parallel, you must wait for the *slowest* (longest) completion before the reward model can score them all and return the best. Response lengths follow a distribution, and the expected maximum of N samples from a distribution is higher than the expected value of a single sample. This right-shift in latency grows with N.

Q15. Choose DPO. Reasons: (a) no RL expertise needed, simpler implementation, only 2 models in memory (fits on 2×A100), 4-week timeline is achievable; PPO would require reward model training + RL tuning under time pressure. Pipeline: start from SFT checkpoint on A100#1 (reference model, frozen) + a copy on A100#2 (trainable policy). Load preference data: 10K pairs, split 80/20 train/val. Format as (x, y_w, y_l) triples. Train with TRL's DPOTrainer, $\beta = 0.1$, QLoRA (rank=16) to fit in memory. Preference data collection: 10K pairs should

be human-labeled with clear rubric (helpfulness, appropriateness, tone). For customer service, dimensions: accurate information, polite tone, appropriate escalation. Evaluation: (a) held-out preference prediction accuracy (does DPO model assign higher implicit reward $\hat{r}(y_w) > \hat{r}(y_l)$ on the val set?), (b) GPT-4-as-judge on 200 held-out prompts comparing SFT vs. DPO outputs, (c) human review of 50 flagged customer service scenarios.

End of Study Guide — Lecture 5

Source: Stanford CME 295, Fall 2025. Afshine & Shervine Amidi.